

More React Concepts

What is Modal?

- UI element that appears **on top of current content**.
- Used for **alerts, confirmations, forms, previews**.
- **Blocks background interaction** until closed.
- **Example:** Login form, Delete confirmation, Image preview.

Modal Best Practices

- Use portals to render outside main DOM.
- Include close buttons and ESC key handling.
- Avoid nested modals to prevent user confusion.

What is Portal?

- Render a component outside parent DOM hierarchy.
- Useful for modals, tooltips, dropdowns, notifications.
- Syntax:

`ReactDOM.createPortal (children, container)`

Portal Example

```
const Modal = ({ children }) => {  
  return ReactDOM.createPortal(  
    <div className="modal">{children}</div>,  
    document.getElementById("modal-root")  
  );  
};
```

```
export default Modal;
```

- **modal-root** is a **div** in **index.html** outside main app root.

Why Use Portal ?

- Avoids CSS overflow / z-index issues.
- Keeps DOM structure clean.
- Enables independent styling and positioning.

What are CSS Modules ?

- Scoped CSS, local to component.
- Prevents class name conflicts.
- File naming: `ComponentName.module.css`.

Using CSS Modules

```
import styles from './Button.module.css';  
const Button = () => <button className={styles.primary}>Click Me</button>;
```

- `.primary` class is **unique in generated HTML**.

CSS Modules Advantages

- Automatic local scoping.
- Works well with React components.
- Can compose classes (allows **inherit styles** from another class):

```
.button { padding: 10px; }  
.primary { composes: button; background: blue; }
```

React Router Data API

Loader and Action

Introduction

- **Loader:** Fetch data before route renders.
- **Action:** Handles form submissions or mutations
 - handles POST/PUT/DELETE logic.
- Used in React Router Data APIs.

Example: Loader

```
import { useLoaderData } from "react-router-dom";

export const loader = async () => {
  const res = await fetch("/api/posts");
  return res.json();
};

const Posts = () => {
  const posts = useLoaderData();
  return <ul>{posts.map(p => <li>{p.title}</li>)}</ul>;
};
```

Example: Action

```
export const action = async ({ request }) => {  
  const formData = await request.formData();  
  const title = formData.get("title");  
  await fetch("/api/posts", { method: "POST", body: JSON.stringify({ title }) });  
  return null;  
};
```

Loader and Action : Benifits

- Keeps data fetching and mutations close to routes.
- Simplifies form handling & error handling.
- Enables declarative routing with data dependencies

Error Boundaries and errorElement in Data API

Error Boundaries in React

- Error boundaries are React components.
- They catch JavaScript errors anywhere in their child component tree.
- They log the errors for monitoring or debugging.
- They display a fallback UI instead of letting the entire React app crash.
- Introduced in React 16 to handle runtime errors gracefully.

Error Boundaries : Key Points

- Catches errors during:
 - Rendering phase, Lifecycle methods, Constructors of child components
- Does not catch errors in:
 - Event handlers, Asynchronous code (like `setTimeout` or `fetch`), Server-side rendering
- Fallback UI:
 - Lets you show a custom UI (like “Something went wrong”) instead of a blank screen or broken app.
- Logging:
 - Can log errors to an external service (e.g., Sentry) for monitoring.

Error Boundaries : Limitation

- Traditional error boundaries must be class components because they rely on the lifecycle methods.
- Functional components do not have lifecycle methods like `componentDidCatch`.
- Therefore, React currently does not support creating full error boundaries with hooks alone.
- You can wrap a class-based error boundary around functional components.
- Some third-party libraries provide hook-based abstractions, but internally they still use class components.

Error Boundaries in React Router

- React Router v6/v7 uses error boundaries for routes: **errorElement** acts as a route-level error boundary.
- When a loader/action/component throws an error:
 1. React Router detects the error.
 2. It renders the nearest ancestor route with an `errorElement`.
- This allows nested routes to handle errors independently, while keeping the rest of the app intact.

What is `errorElement` ?

- A property defined on a route in a data router (`createBrowserRouter`, `createMemoryRouter`, etc.)
- Specifies a **React component** to render **instead of the normal element** when something goes wrong:
 1. Loader throws an error
 2. Action fails
 3. Component throws during render
- Access the error using `useRouteError()`

errorElement : Key Behavior

1. Error Bubbling:

- If a route has no errorElement, error bubbles to nearest ancestor route with errorElement.

2. Thrown Types:

- Can throw Response objects (HTTP-aware) or plain JS errors

3. Default Error Element:

- If no ancestor has errorElement, React Router uses a default red-box error UI

4. Helpers:

- **isRouteErrorResponse** helps detect Response objects in error boundaries

Why `errorElement` ?

- Provides route-level error boundaries
- Supports HTTP-aware errors with `Response`
- Works with nested routes and error bubbling
- Can define global or granular error handling

Custom Hooks

What are Custom Hooks?

- **Definition:** A JavaScript function whose name starts with "use" and calls other React hooks.
- **Purpose:** To abstract and share stateful logic between multiple components without duplicating code.
- **Think of them as:** Building blocks for reusable logic. They are not components themselves, they are functions that give your components powerful functionality.

Why use Custom Hooks?

- **Reusability:** Write complex logic once and use it in many different components.
- **Cleaner Components:** Separate the "**what to render**" (UI) from the "**how it works**" (logic). This makes your components smaller and easier to read.
- **Readability:** Encapsulate complex logic in a single, well-named function, making the component's code clear.

Rules of Custom Hooks?

1. **Name Convention** - The name must start with **use**. This allows the React linter to check for hook-related bugs.
2. **Top-Level Only** - Only call a custom hook at the **top level** of a component or another custom hook. Do not call them inside loops, conditions, or nested functions.

Example 1: useCounter hook

- **Problem:** You need a counter in multiple components.
- **Solution:** Create a custom useCounter hook.

```
import { useState } from 'react';

const useCounter = (initialValue = 0) => {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(prevCount => prevCount + 1);
  const decrement = () => setCount(prevCount => prevCount - 1);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset };
};
```

Example 1: useCounter hook

- **How it works:** Call the hook and use its returned values. Each component using the hook gets its own isolated state.

```
import useCounter from './useCounter';

const CounterComponent = () => {
  // Each component gets its own 'count' state
  const { count, increment, decrement, reset } = useCounter(10);

  return (
    <div>
      <h3>Count: {count}</h3>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
};
```

Example 2: useFetch hook

- **Problem:** You need to fetch data and manage loading/error states in multiple places.
- **Solution:** Create a useFetch custom hook.

```
import { useState, useEffect } from 'react';

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(url);
        if (!response.ok) throw new Error('Network response was not ok');
        const result = await response.json();
        setData(result);
      } catch (e) {
        setError(e);
      } finally {
        setLoading(false);
      }
    };
    fetchData();
  }, [url]);

  return { data, loading, error };
};
```

Example 2: useFetch hook

- **How it works:** The hook handles the entire data fetching lifecycle, leaving your component clean and simple.

```
import useFetch from './useFetch';

const UserList = () => {
  const { data, loading, error } = useFetch('https://api.example.com/users');

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;

  return (
    <div>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </div>
  );
};
```

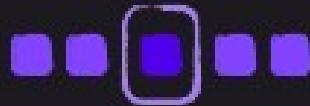
Redux

A State Management Library

What is Redux?

- A state management library for JavaScript apps
- Commonly used with React, but works with any framework
- Provides a centralized store for app state

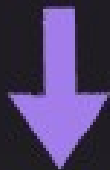
What Is Cross-Component & App-Wide State?



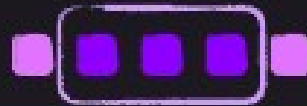
Local State

State belongs to a single component

E.g., listening to user input on an input field or toggling a "show more details" field



Should be **managed inside the component** via `useState()` / `useReducer()`



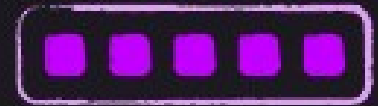
Cross-component State

State affecting multiple components

E.g., open / closed state of a modal overlay



Requires "**prop drilling**"



App-wide State

State affecting the entire app

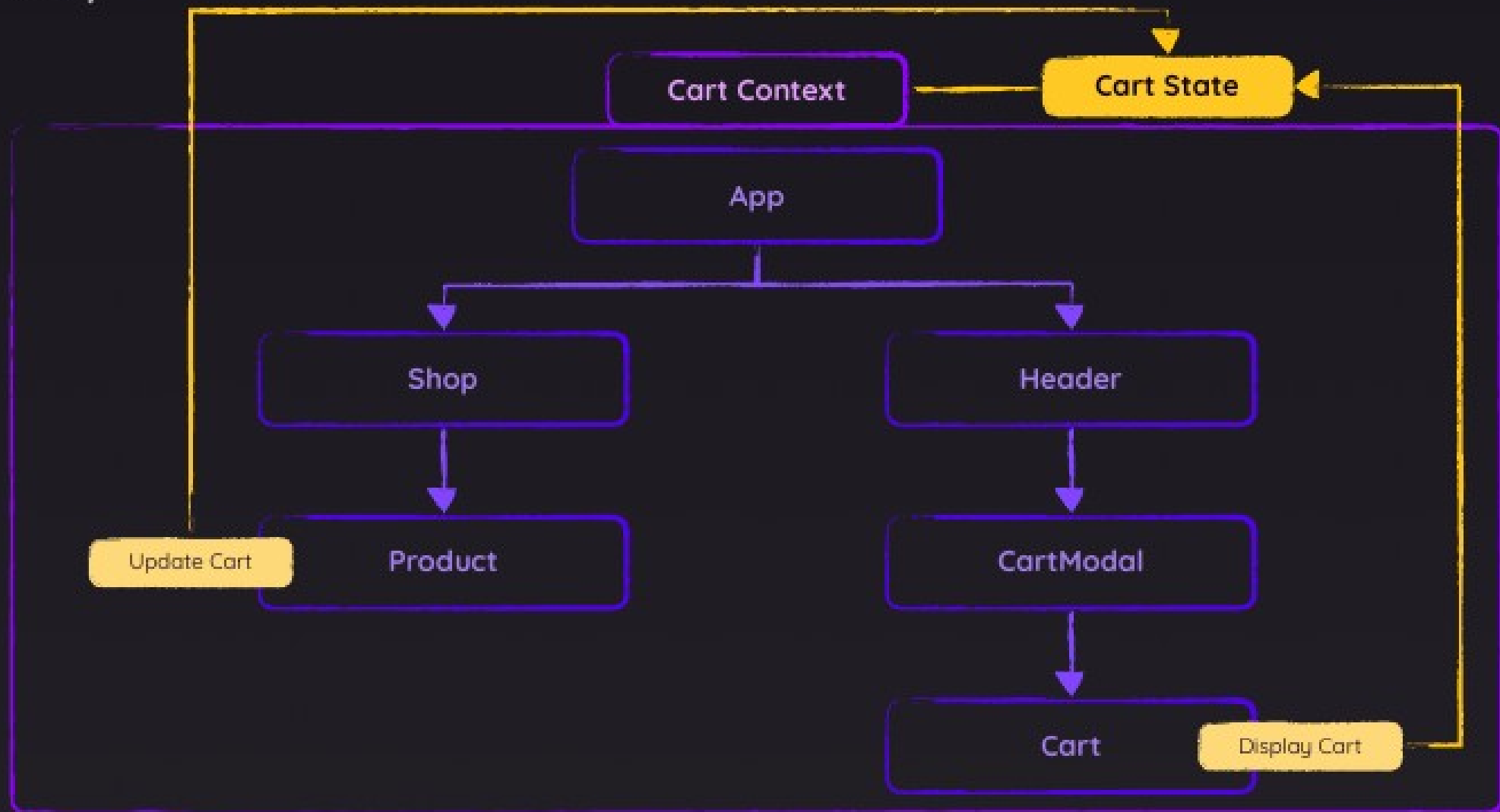
E.g., user authentication status or chosen theme



Requires "**prop drilling**"

Or: **React Context** or **Redux**

React Context Manages Multi-Component State



**A state management
system for cross-component
or app-wide state**

Don't we have React Context already?

React Context Has Some Potential Disadvantages



Complex Setup & Management

In more complex apps, using React Context can lead to deeply nested or “fat” “Context Provider” components



Performance

React Context is not optimized for high-frequency state changes

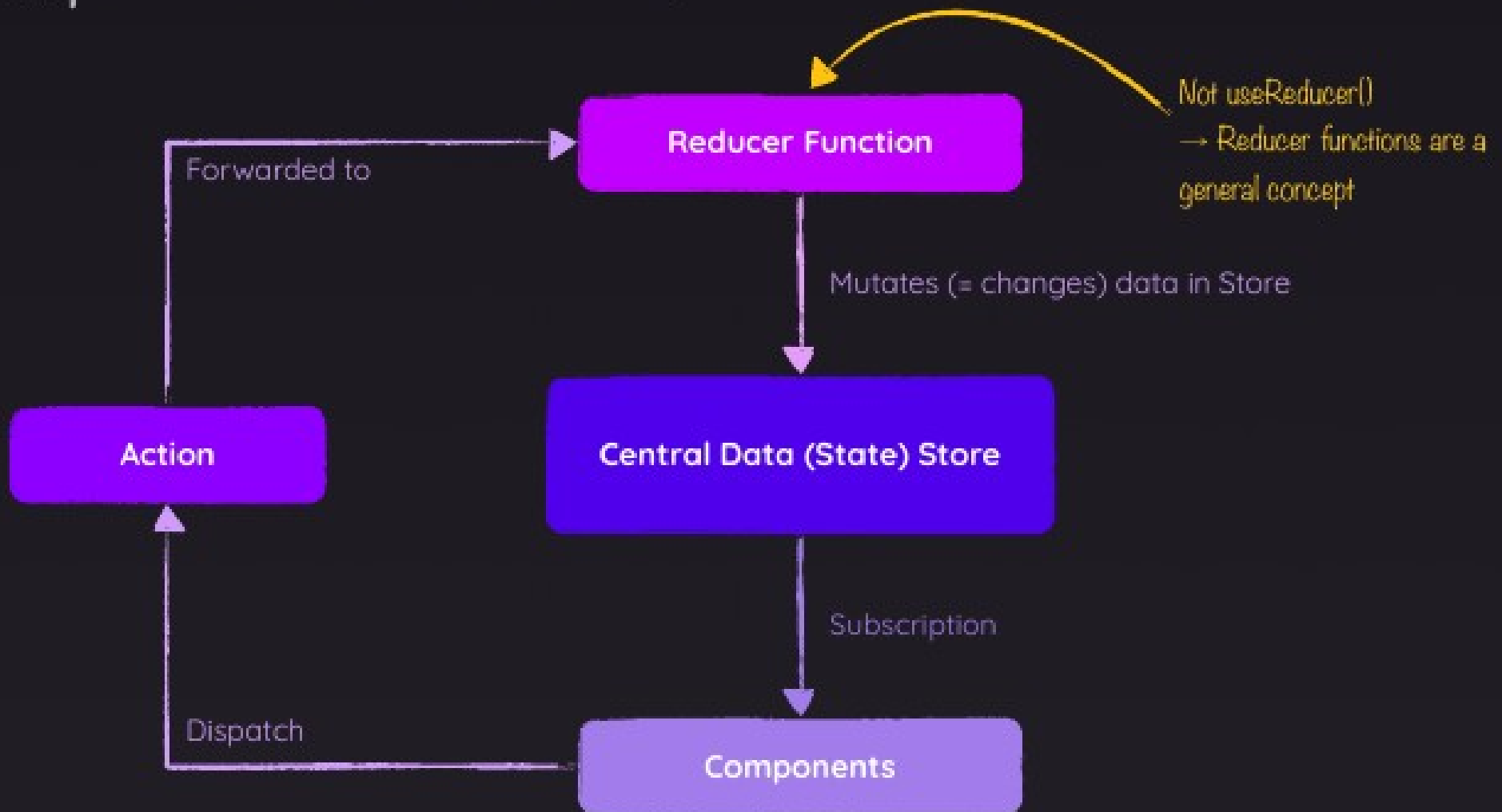
Potential Problem: Deeply Nested Providers

```
return (  
  <AuthContextProvider>  
    <ThemeContextProvider>  
      <UIInteractionContextProvider>  
        <MultiStepFormContextProvider>  
          <UserRegistration />  
        </MultiStepFormContextProvider>  
      </UIInteractionContextProvider>  
    </ThemeContextProvider>  
  </AuthContextProvider>  
);
```

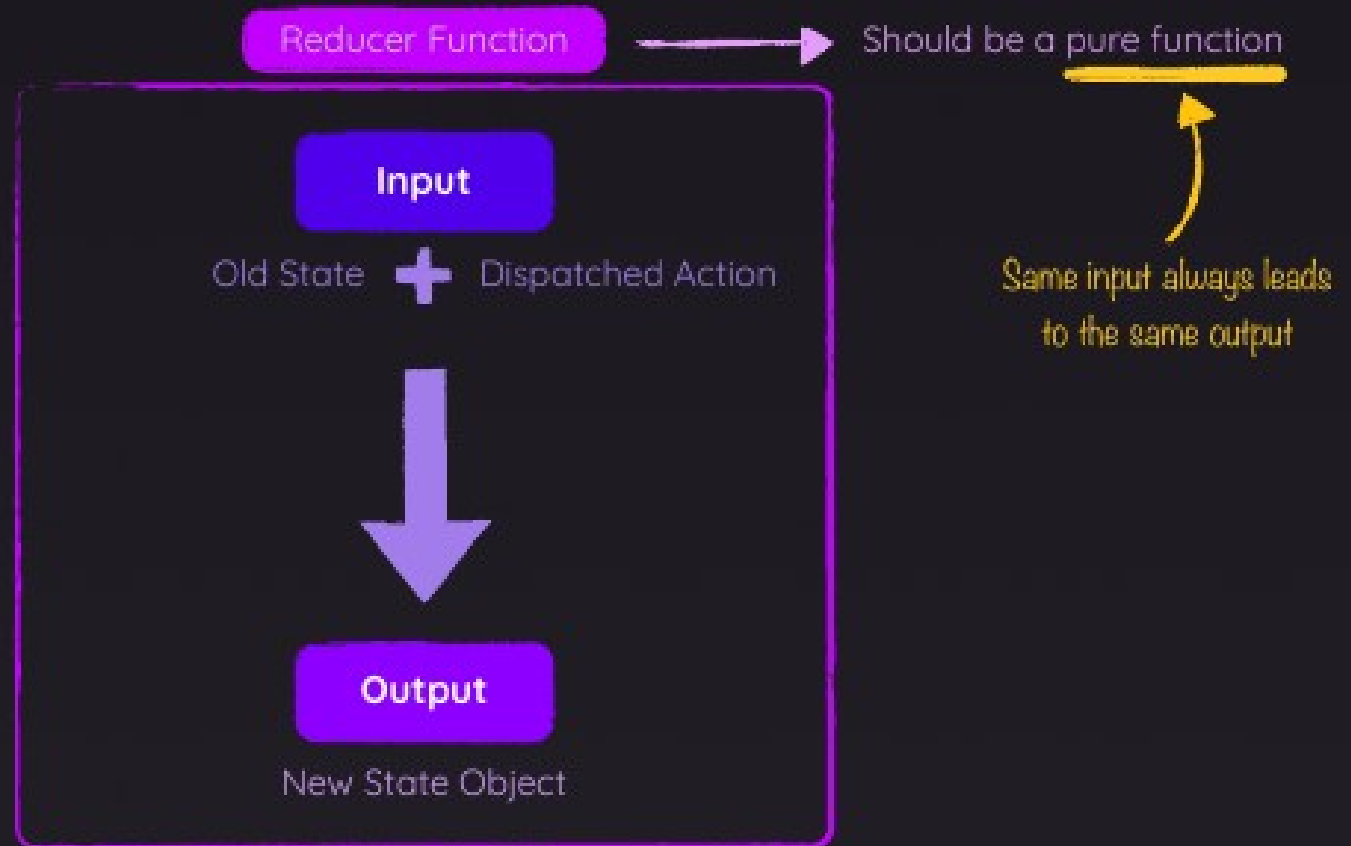
Potential Problem: Complex Providers

```
function AllContextProvider() {  
  const [isAuth, setIsAuth] = useState(false);  
  const [isEvaluatingAuth, setIsEvaluatingAuth] = useState(false);  
  const [activeTheme, setActiveTheme] = useState('default');  
  const [ ... ] = useState(...);  
  
  function loginHandler(email, password) { ... };  
  
  function signupHandler(email, password) { ... };  
  
  function changeThemeHandler(newTheme) { ... };  
  
  ...  
  
  return (  
    <AllContext.Provider>  
  
    </AllContext.Provider>  
  )  
}
```

Core Redux Concepts



The Reducer Function



Core Principles

- Single Source of Truth – one central store
- State is Read-only – updated only via actions
- Changes with Pure Functions – done using reducers

Key Concepts

- **Store** → holds application state
- **Action** → plain object { type: "ACTION_NAME" }
- **Reducer** → pure function (state, action) → newState
- **Dispatch** → sends actions to reducers
- **Selector** → retrieves data from state

Why Redux?

- Handles complex state in large apps
- Ensures predictable state updates
- Easy debugging with time-travel
- Supports async with middlewares (Redux Thunk, Saga)

Simple Redux Flow

- Dispatch Action → { type: "INCREMENT" }
- Reducer Updates State → returns new state
- Store Notifies UI → UI re-renders with updated data